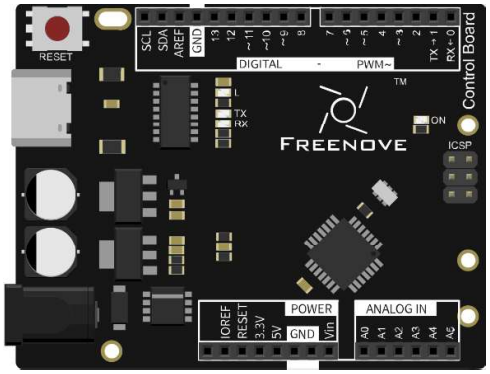
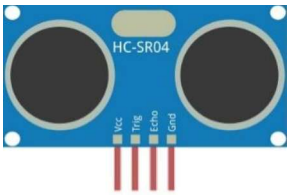
Chapter 14 Ultrasonic Ranging

In this chapter, we learn a module that use ultrasonic to measure distance, HC-SR04 ultrasonic ranging module.

Project 14.1 Ultrasonic Ranging

In this project, we use ultrasonic ranging module to measure distance, and print out the data in the serial port.

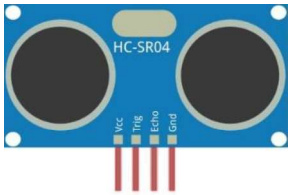
Component List

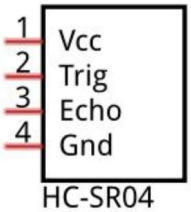
Control board x1 	USB cable x1
	Jumper F/M x4
	Ultrasonic ranging module x1 

Component Knowledge

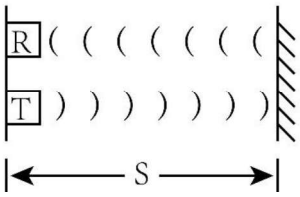
Ultrasonic ranging module

The HC-SR04 Ultrasonic Ranging Module integrates a both an ultrasonic transmitter and a receiver. The transmitter is used to convert electrical signals (electrical energy) into high frequency (beyond human hearing) sound waves (mechanical energy) and the function of the receiver is opposite of this. The picture and the diagram of the HC SR04 Ultrasonic Ranging Module are shown below:





The Ultrasonic Ranging Module uses the principle that ultrasonic waves will reflect when they encounter any obstacles. This is possible by counting the time interval between when the ultrasonic wave is transmitted to when the ultrasonic wave reflects back after encountering an obstacle. Time interval counting will end after an ultrasonic wave is received, and the time difference (delta) is the total time of the ultrasonic wave's journey from being transmitted to being received. Because the speed of sound in air is a constant, and is about v=340m/s, we can calculate the distance between the Ultrasonic Ranging Module and the obstacle: s=vt/2.



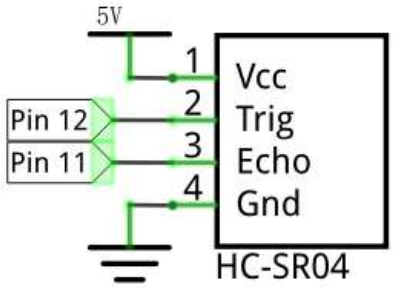
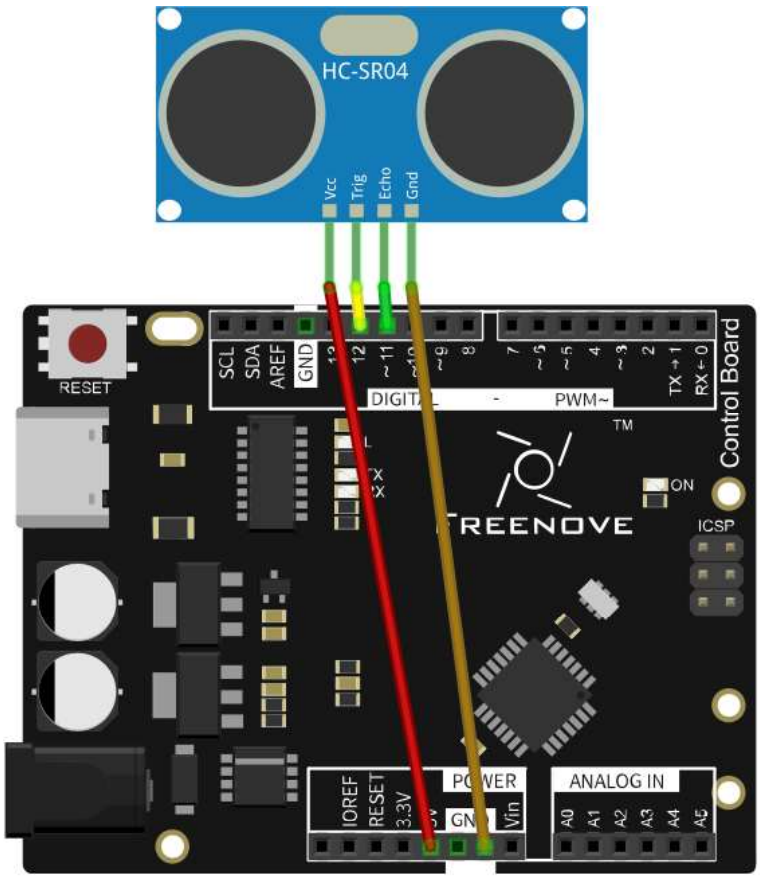
Pin description:

Pin name	Pin number	Description
Vcc	1	Positive electrode of power supply, the voltage is 5V
Trig	2	Triger pin
Echo	3	Echo pin
Gnd	4	Negative electrode of power supply

Instructions for use: output a high-level pulse in Trig pin lasting for least 10uS, the module begins to transmit ultrasonic waves. At the same time, the Echo pin is pulled up. When the module receives the returned ultrasonic waves from encountering an obstacle, the Echo pin will be pulled down. The duration of high level in the Echo pin is the total time of the ultrasonic wave from transmitting to receiving, s=vt/2. This is done constantly.

Circuit

The connection of the control board and HC-SR04 is shown below.

Schematic diagram 
Hardware connection 

Sketch

Sketch 14.1.1

First, we use the HC-SR04 communication protocol to operate the module, get the range of time, and calculate the distance.

```
#define trigPin 12 // define TrigPin
#define echoPin 11 // define EchoPin.
#define MAX_DISTANCE 200 // Maximum sensor distance is rated at 400-500cm.
// define the timeOut according to the maximum range. timeOut= 2*MAX_DISTANCE /100 /340 *1000000 = MAX_DISTANCE*58.8
float timeOut = MAX_DISTANCE * 60;
int soundVelocity = 340; // define sound speed=340m/s

void setup() {
  pinMode(trigPin,OUTPUT);// set trigPin to output mode
  pinMode(echoPin,INPUT); // set echoPin to input mode
  Serial.begin(9600);    // Open serial monitor at 9600 baud to see ping results.
}

void loop() {
  delay(100); // Wait 100ms between pings (about 20 pings/sec). 29ms should be the shortest delay between pings.
  Serial.print("Ping: ");
  Serial.print(getSonar()); // Send ping, get distance in cm and print result (0 = outside set distance range)
  Serial.println("cm");
}

float getSonar() {
  unsigned long pingTime;
  float distance;
  digitalWrite(trigPin, HIGH); // make trigPin output high level lasting for 10μs to triger HC_SR04,
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  pingTime = pulseIn(echoPin, HIGH, timeOut); // Wait HC-SR04 returning to the high level and measure out this waitting time
  distance = (float)pingTime * soundVelocity / 2 / 10000; // calculate the distance according to the time
  return distance; // return the distance value
}
```

First, define the pins and the maximum measurement distance.

```
#define trigPin 12 // define TrigPin
#define echoPin 11 // define EchoPin.
#define MAX_DISTANCE 200 // Maximum sensor distance is rated at 400-500cm.
```

If the module does not return high level, we cannot wait for this forever. So we need to calculate the the time period for the maximum distance, that is, time Out. timOut= 2*MAX_DISTANCE/100/340*1000000. The result of the constant part in this formula is approximately equal to 58.8.

```
float timeOut = MAX_DISTANCE * 60;
```

Then, in the setup (), set the pin to input or output, and set the serial port. In the loop(), we continue to use serial to print the value of subfunction getSonar (), which is used to return the measured distance of the HC_SR04. Make trigPin output a high level lasting for at least 10μs, according to the communication protocol.

```
  digitalWrite(trigPin, HIGH); // make trigPin output high level lasting for 10μs to triger HC_SR04,
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
```

And then the echoPin of HC_SR04 will output a pulse. Time of the pulse is the total time of ultrasonic from transmitting to receiving. We use the pulseIn () function to return the time, and set the timeout.

```
  pingTime = pulseIn(echoPin, HIGH, timeOut); // Wait HC-SR04 returning to the high level and measure out this waitting time
```

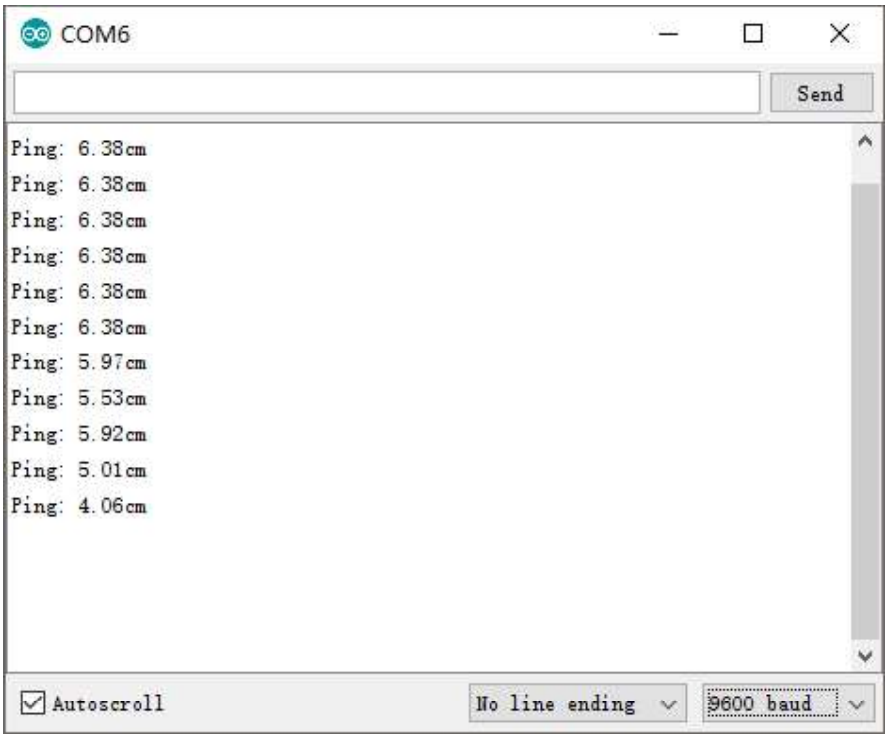
Calculate the distance according to the time and return the value.

```
  distance = (float)pingTime * soundVelocity / 2 / 10000; // calculate the distance according to the time
  return distance; // return the distance value
```

Finally, the code above will be called in the loop ().

pulseIn(pin, value) / pulseIn(pin, value, timeout)
Return the length of the pulse (in microseconds) or 0 if no pulse is completed before the timeout (unsigned long).

Verify and upload the code to the control board, open the serial port monitoring window, turn the HC-SR04 probe towards the object plane, and observe the data in the serial port monitoring window.


COM6
Send
Ping: 6.38cm
Ping: 6.38cm
Ping: 6.38cm
Ping: 6.38cm
Ping: 6.38cm
Ping: 6.38cm
Ping: 5.97cm
Ping: 5.53cm
Ping: 5.92cm
Ping: 5.01cm
Ping: 4.06cm
Autoscroll
No line ending
9600 baud